



BuildX

AUTONOMOUS MARKETING

UI / UX Demo Design

The console for a hierarchical, multi-tenant marketing agent system that runs campaigns with no human in the loop.

NO HUMAN IN THE LOOP

MULTI-TENANT

LIVE CAPTURES

Every screen in this deck is a real capture of the running application — not a mockup.

github.com/apramitroy1631-ship-it/Midas

01 / 04

Design System

Tokens, type, and the four principles the rest of the deck keeps coming back to.

Design Principles

The visual language is a direct consequence of what the product has to prove.

Dark-first, not dark-only

This is a monitoring surface people leave open on a second monitor for hours. Dark reduces eye strain over a long session; a full light-theme token swap exists for daytime use (the Theming screen, later in this deck), not an inverted hack.

Status is color, always paired with text

Green/amber/red never carry meaning alone — every status dot sits beside a word (“published”, “blocked”, “abandoned”). Color-blind users and screenshots in Slack both still read correctly.

Monospace marks anything the system generated

IDs, timestamps, scores, costs — machine output — render in mono. Prose the agents or a person wrote renders in the sans body font. The split lets your eye sort “data” from “writing” without reading either.

The autonomy badge is never absent

“No human in the loop” sits in the top bar on every single screen, not just the launch view. The one fact a stakeholder must never lose track of is structurally impossible to scroll past.

Color & Type

The shipped app's own tokens — `web/src/styles.css`, unchanged by this deck's `cmartsolutions.com`-matched cover and chrome.



`--bg`
page ground



`--surface`
cards



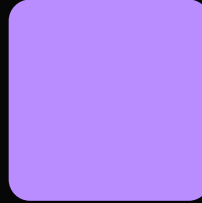
`--surface-2`
inputs / raised



`--border-soft`
hairlines



`--accent`
primary / links



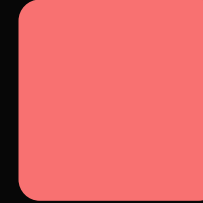
`--accent-2`
secondary tenant



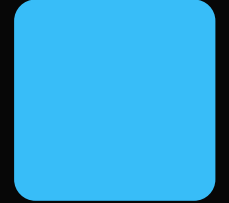
`--ok`
published / passed



`--warn`
revision / advisory



`--danger`
abandoned / critical



`--running`
in progress

Typography (app)

Aa

Sans — UI chrome, prose, agent-written copy
System sans-serif stack, no webfont dependency

01:23

Mono — IDs, timestamps, scores, costs, code
Anything the system generated, never hand-written copy

Spacing & radius

- Base radius 10px (cards), 6px (controls) — soft, not skeuomorphic
- 1px hairline borders throughout — depth comes from color, not shadow
- Consistent 14–22px internal card padding, no bespoke spacing per view

02 / 04

Getting In

Connecting to a tenant, and the first thing the operator sees.

BX

BuildX Console

AUTONOMOUS MARKETING

Connect with a tenant API key. Each key sees only its own tenant's brands, runs, and assets — connect several and switch between them from the top bar.

API base URL

http://127.0.0.1:8100

Tenant API key

bx_live_.....

Shown once when the tenant was provisioned. Stored in this browser only.

Connect

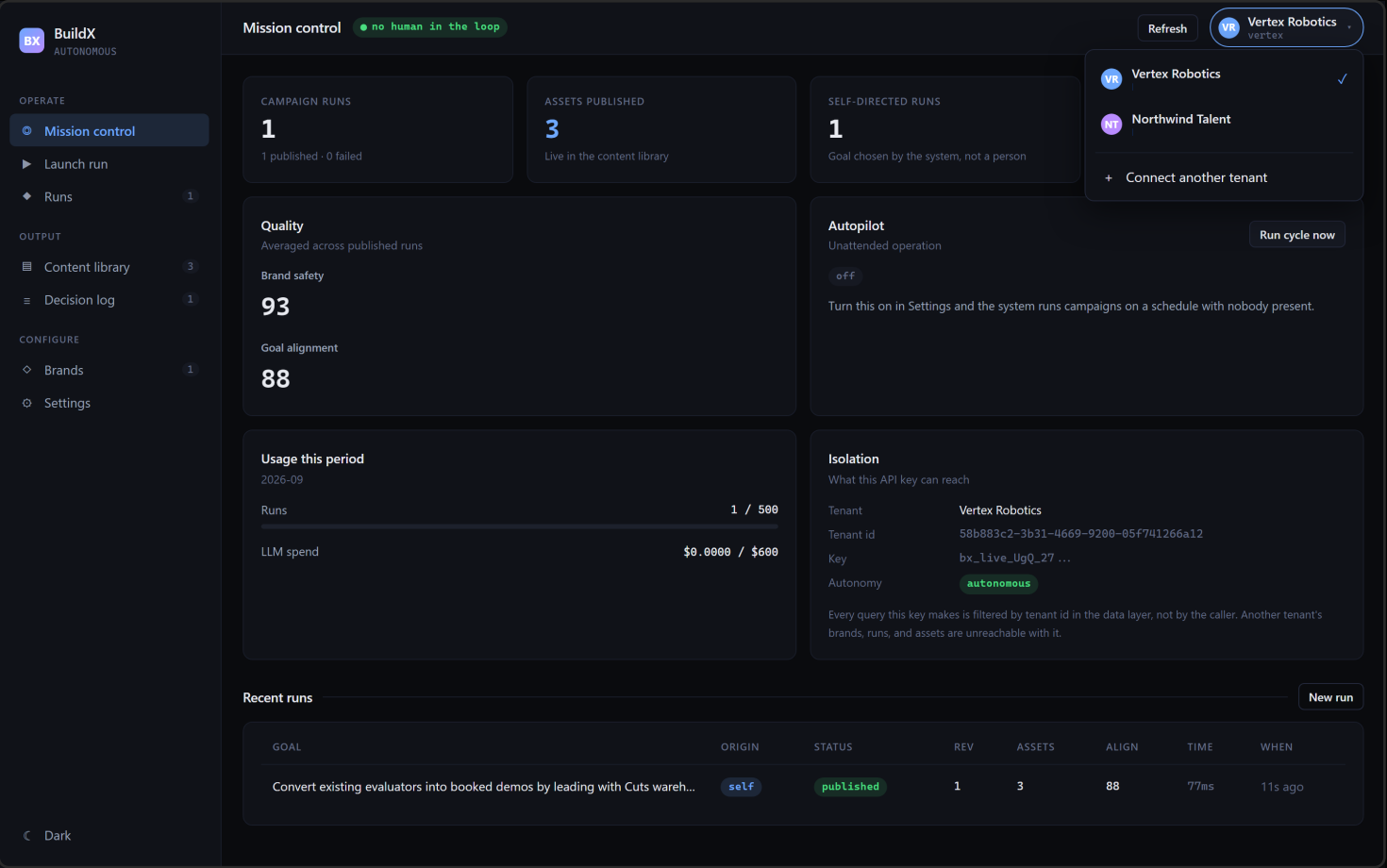
Why it's built this way

- **One credential, one tenant**
There's no login/signup flow to design — a tenant API key is the entire authentication model, verified against /v1/me before it's ever saved, so a bad key fails right here instead of three screens later.
- **Keys live in this browser only**
Never sent anywhere but the API this browser is pointed at, and never logged. The hint text says so explicitly, because a demo audience's first question is always “where does this go?”
- **Several tenants, one shell**
Connecting a second key doesn't replace the first — it adds an entry to a switcher (see the Mission Control screen next), which is what makes the multi-tenant claim demonstrable rather than asserted.

03 / 04

Operating the System

Mission control, launching a run, and watching the self-correction loop happen live.



Why it's built this way

- **The autonomy badge, always present**
“No human in the loop” sits in the top bar on every screen in the product, not just this one — the fact that governs everything else is structurally impossible to lose track of.
- **Isolation stated, not implied**
The Isolation card names this exact tenant id and key prefix and says plainly what it can and can't reach. That sentence is true because the data layer enforces it, not because the UI asks nicely.
- **Quality is a number, not a vibe**
Brand safety and goal alignment are QA's own scores, averaged across real runs — the dashboard reports what the agents concluded about their own work, unfiltered.

BX

BuildX

AUTONOMOUS

OPERATE

Mission control

Launch run

Runs

OUTPUT

Content library

Decision log

CONFIGURE

Brands

Settings

Dark

Launch an autonomous run

no human in the loop

Refresh

Vertex Robotics

Campaign brief

Everything here is optional except the brand.

Brand

Vertex Robotics

Campaign goal

e.g. Drive demo requests for the Q4 launch

Leave blank and the Campaign Director reads brand memory and picks the goal itself.

Audience override

Operations directors at 200-2000 headcot

Optional. The Director sharpens this either way.

Budget (USD)

8000

Launch autonomous run

Autonomy

fully autonomous

QA failures loop back to the writer instead of stopping for a person. Content publishes on a clean pass with no human approval.

Revision budget

2 cycles

Runs this period

0 / 500

Ready to run

Eleven agent steps: the Director plans and delegates, specialists research, strategise, write, optimise and review, and QA decides whether it ships. A blocked draft comes back here as a revision cycle, not a request for your approval.

Why it's built this way

- **The goal field is optional on purpose**

Leave it blank and the Campaign Director reads brand memory and picks one itself — the empty state's hint text says exactly this, so the self-directed mode isn't a hidden feature.

- **Autonomy is explained at the point of use**

The right-hand panel restates what “fully autonomous” means for this specific run, right where the operator is about to trigger one — not buried in a settings page they'd need to go find.

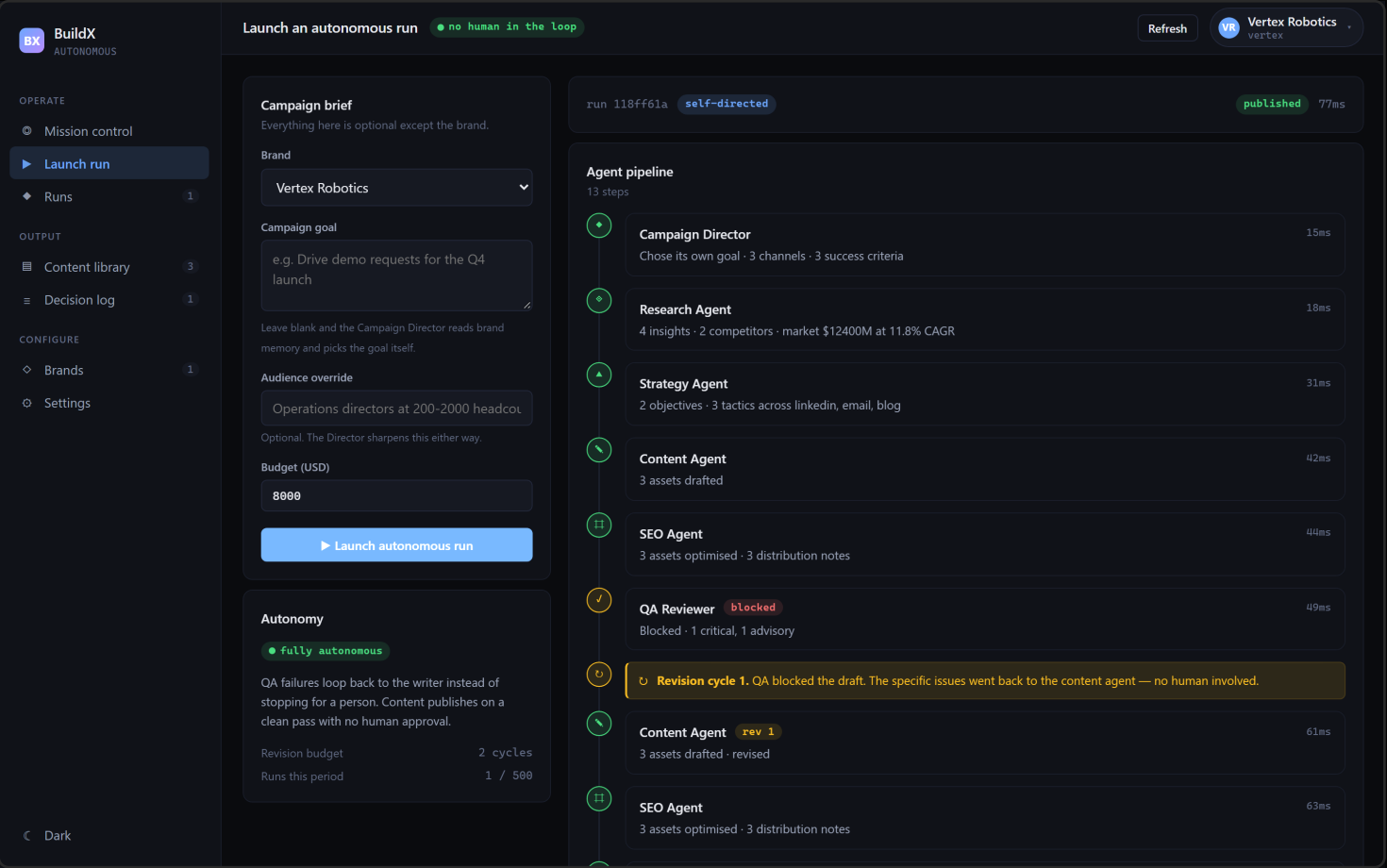
- **Revision budget is visible before you commit**

Seeing “2 cycles” here, not after a run fails, sets the right expectation for what “autonomous” is actually going to do if something goes wrong.

BUILDX — UI/UX DEMO DESIGN

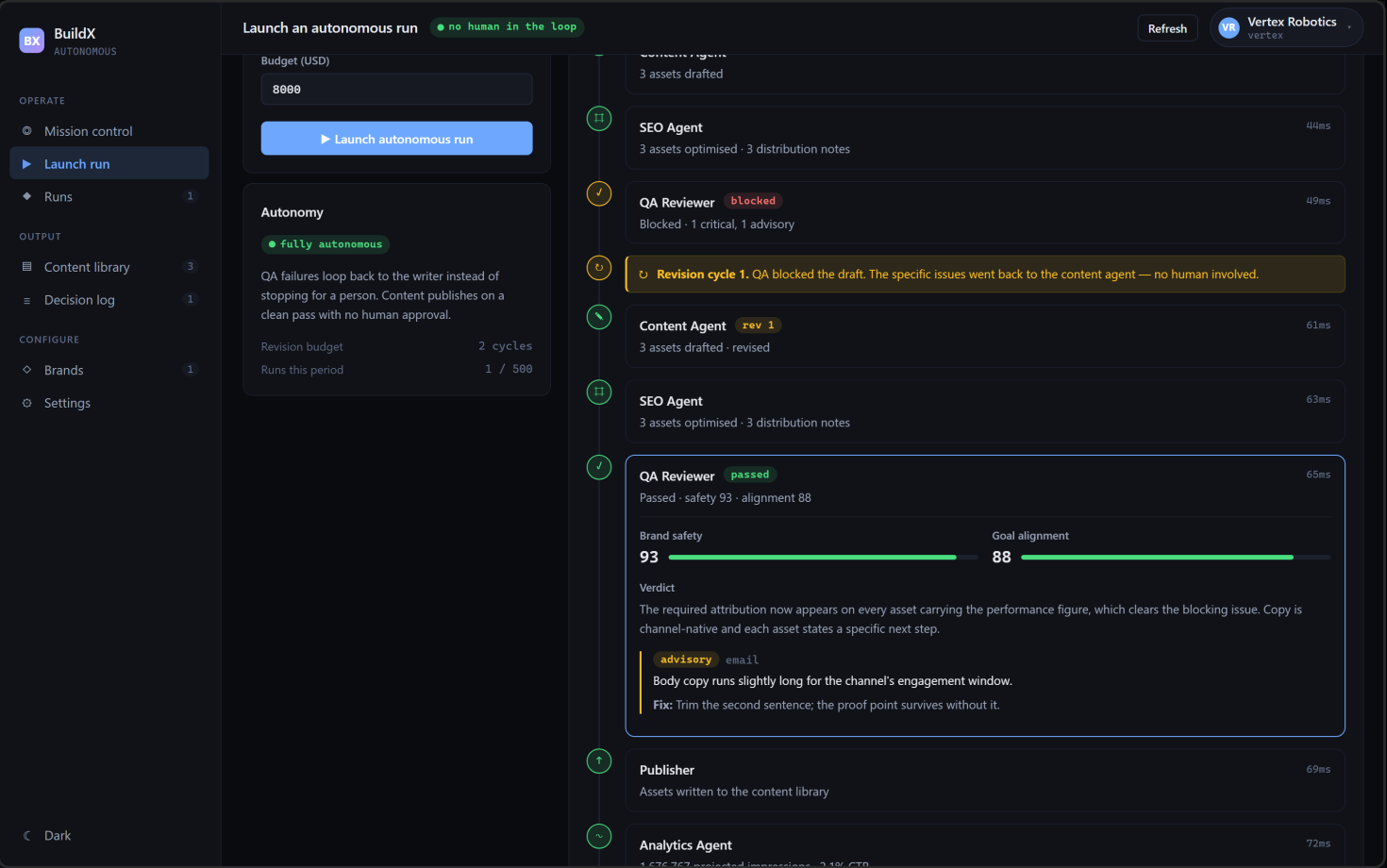
LAUNCH — BRIEF

09



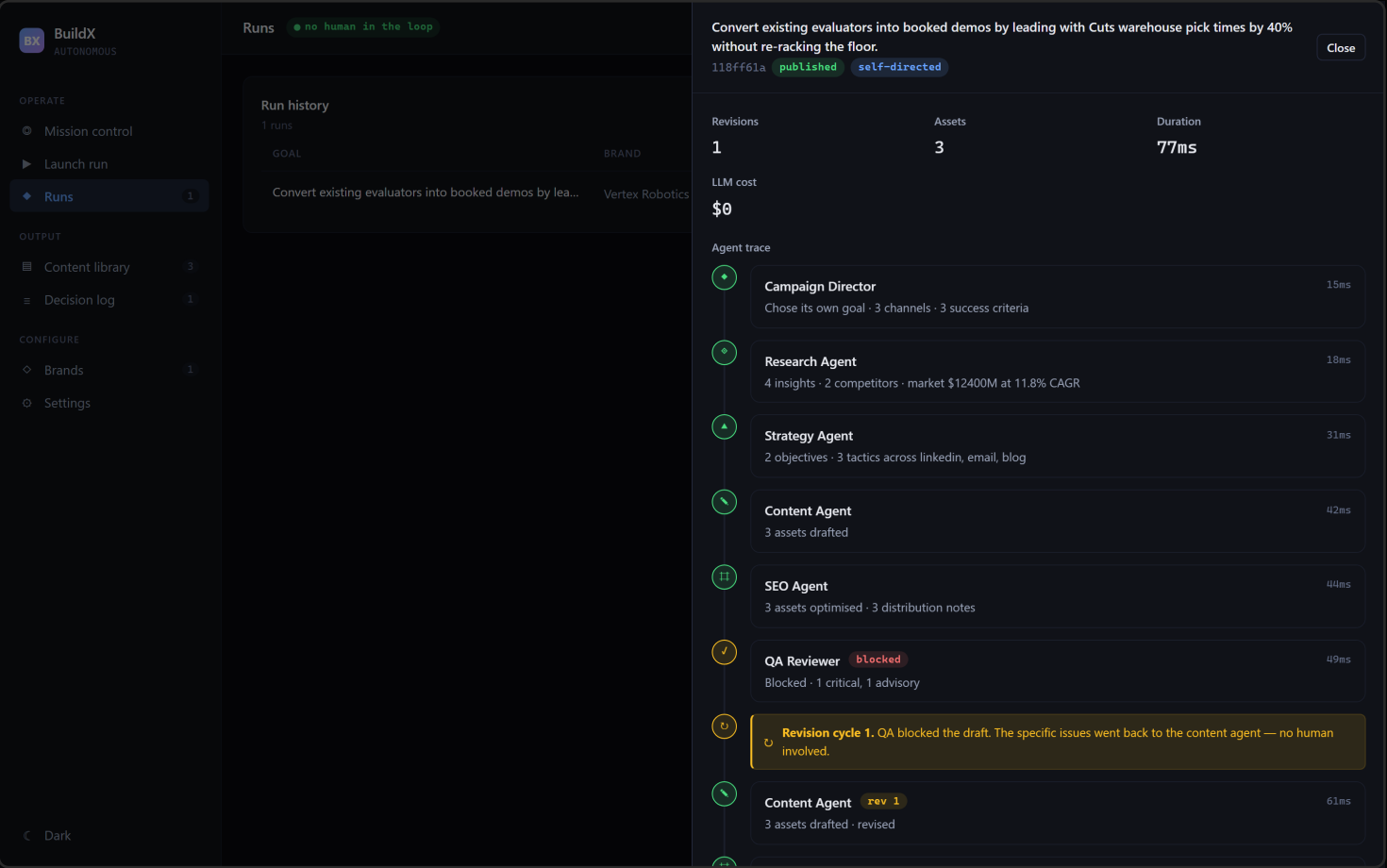
Why it's built this way

- **The single most important element in the product**
QA blocked the first draft; the revision banner shows the specific issues going back to the writer — with no human involved. This is the moment that would normally need a person, made visible instead of invisible.
- **Every node is a real decision, not a spinner**
Each step summarises what that agent actually concluded — 4 insights, 2 competitors, a specific market figure — while it's still running, via the SSE stream. Nothing here is a fake progress bar.
- **Revision cycles render as their own step**
Rather than silently re-running the content agent, “revise” gets its own row in the timeline — the loop is a visible event, not something you'd only find by reading logs.



Why it's built this way

- **Click any step, see the actual agent output**
This is the QA agent's real structured response — scores, verdict, and every issue with its channel and a concrete fix — not a generic “completed” state.
- **Severity is explicit, never inferred from color alone**
Each issue is labelled critical or advisory in text next to the color, because a screenshot shared in Slack shouldn't need the reader to know the palette.
- **The meter bars make the score visceral**
93 and 88 read fine as numbers, but the filled bar next to each one is what makes “this passed comfortably” legible in half a second.



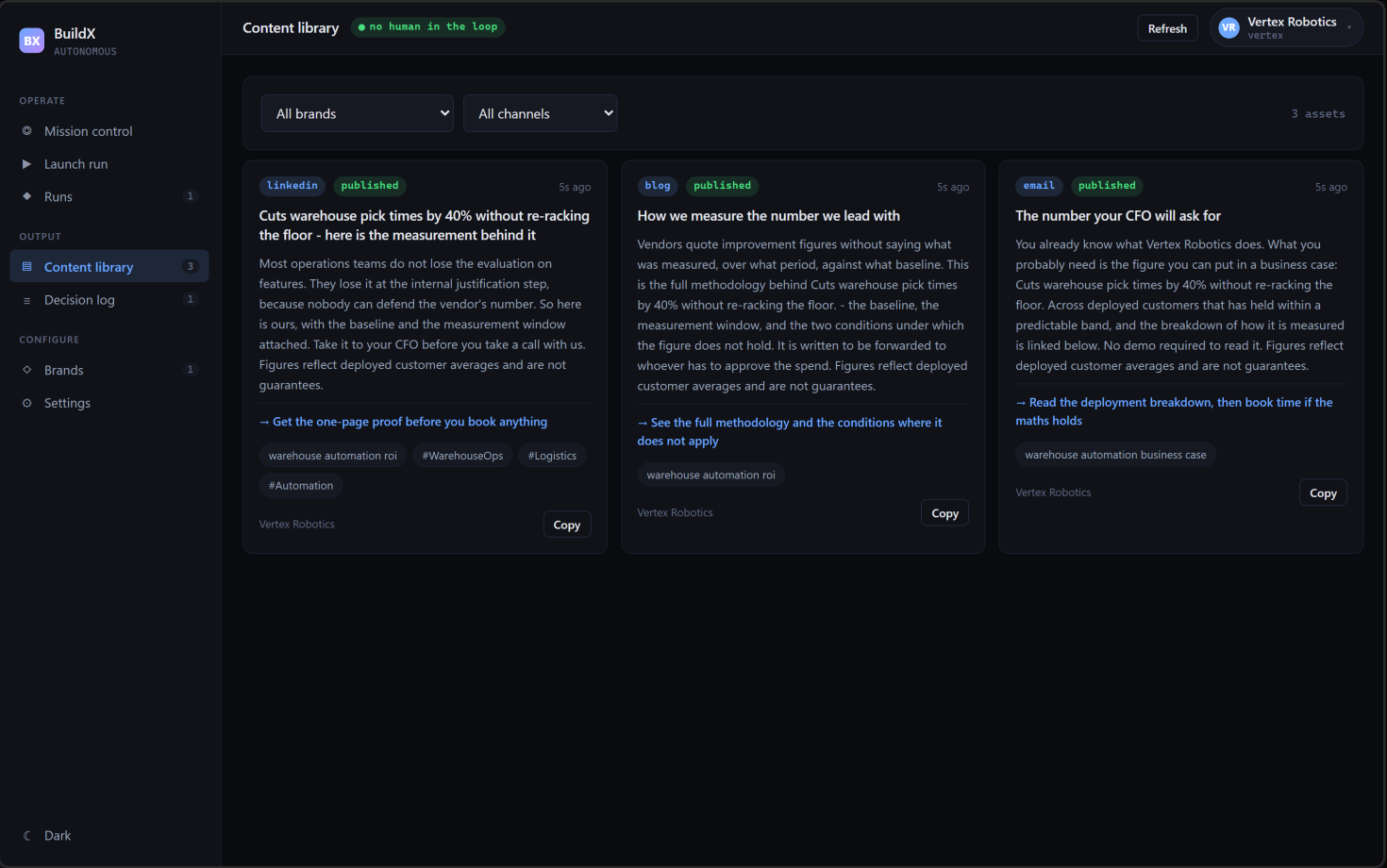
Why it's built this way

- **Every run is replayable after the fact**
The drawer shows the identical agent trace from the live view — nothing about a run's history is summarised away once it's over.
- **A drawer, not a new page**
Opening a run's detail doesn't lose the list behind it — the table stays visible and dimmed, so comparing across runs doesn't mean losing your place.
- **Origin badge travels with the run permanently**
“self-directed” vs an operator-supplied goal stays attached to the run forever, not just at launch time — it's a fact about the decision, worth keeping.

04 / 04

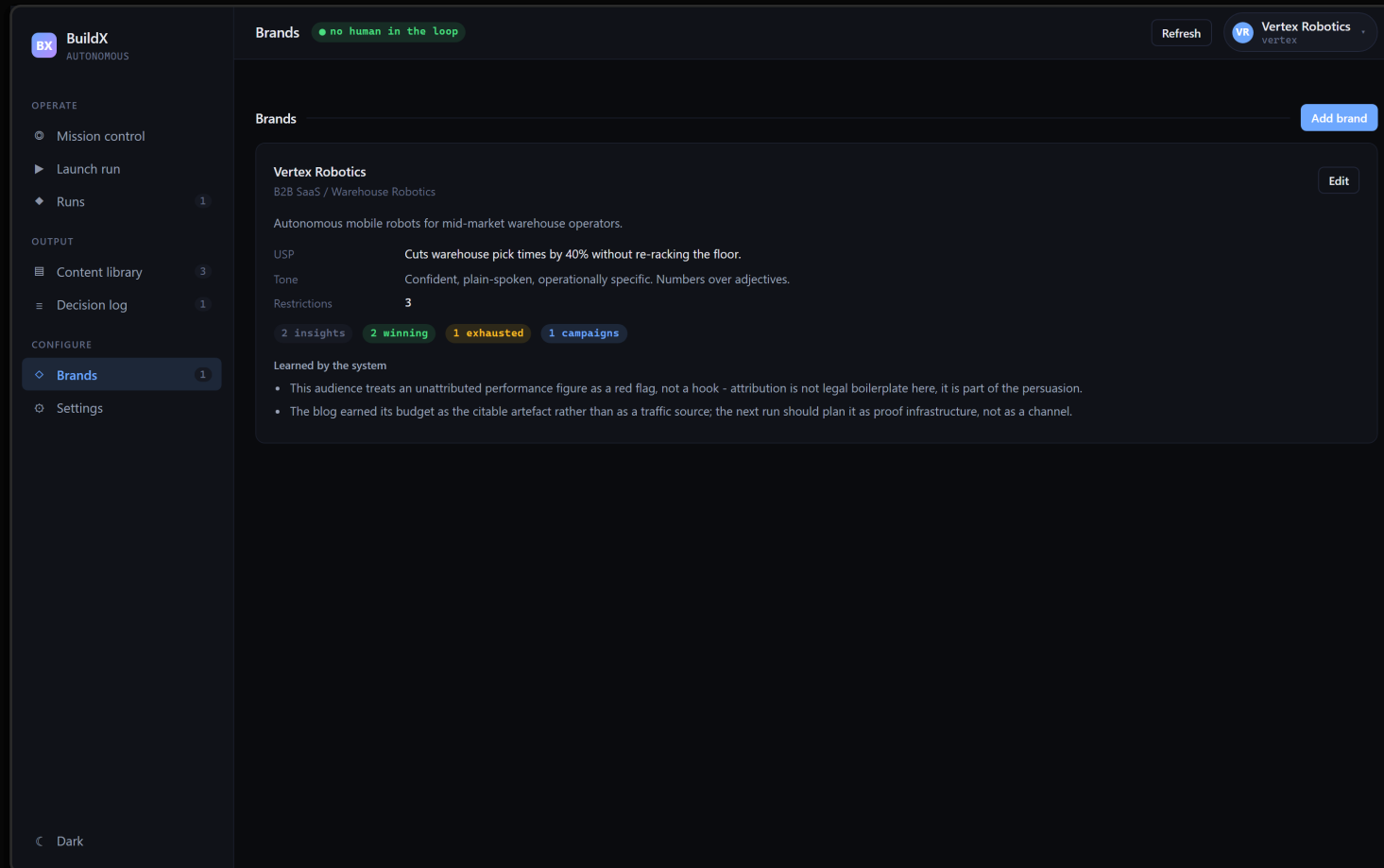
What the System Produces

The content library, brand memory, the decision log, and the settings that decide how much rope the agents get.



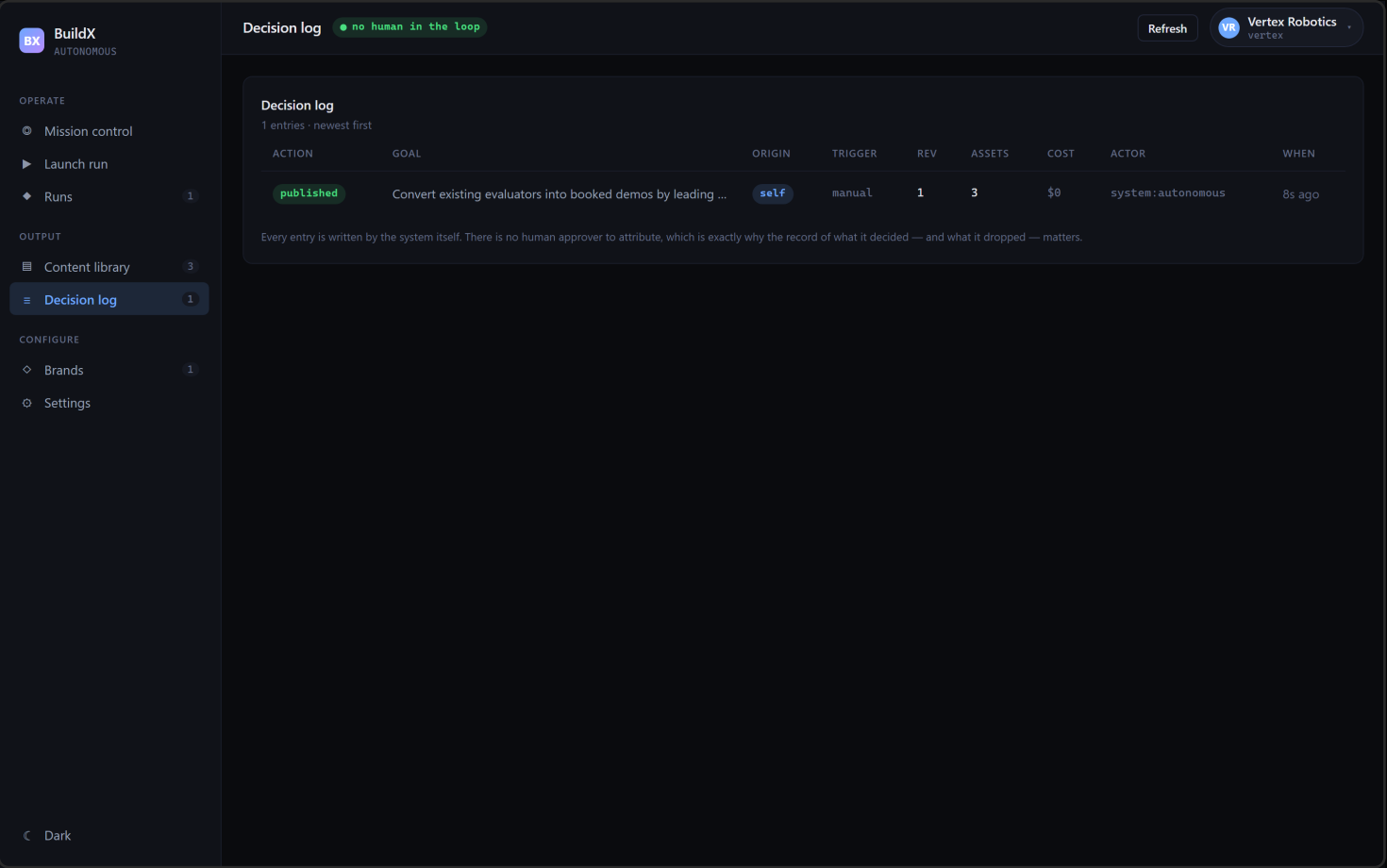
Why it's built this way

- Published, not pending — by default
Assets land here the moment QA passes them, tagged with their real status. Nothing sits in an approval queue unless the tenant's policy explicitly asks for that.
- Copy stays exactly as the agent wrote it
Headline, body, and CTA render verbatim, with SEO keywords and hashtags surfaced as chips — the library is meant to be copy-pasted straight into a real channel.
- Filterable the way a marketer actually thinks
By brand, then by channel — the two dimensions someone managing several tenants' output would actually need to slice by.



Why it's built this way

- **The compounding loop, made visible**
“Learned by the system” shows the actual insights the last run wrote back — this is what lets the next run avoid repeating an angle, not a claim in a README.
- **Winning vs. exhausted, counted separately**
Two different badges, because they mean opposite things for planning: one says “do more of this,” the other says “this is used up, don't replan around it.”
- **Restrictions are the one non-negotiable field**
Content restrictions are what QA treats as absolute — they're never optional in the edit drawer, because a breach here is what actually blocks a publish.

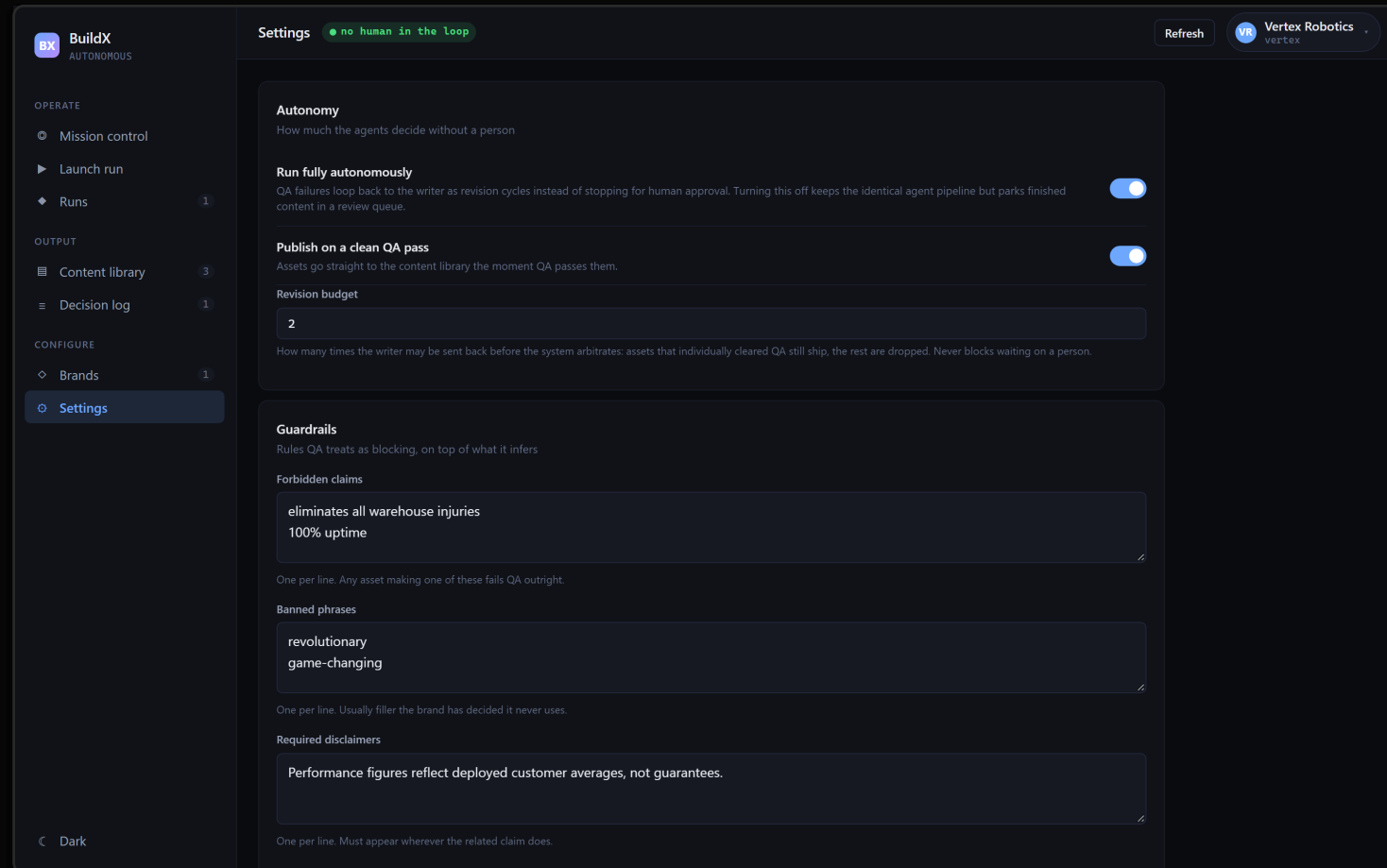


Why it's built this way

- **The record that replaces a human approver**
Nobody signs off on a run, so every outcome — published, dropped, abandoned — is written here with a reason. This table is the accountability structure.
- **Cost sits next to the decision it paid for**
LLM spend is attributed per run right in this log, not hidden in a separate billing view — autonomy and cost are the same conversation.
- **Self-directed runs are flagged, not hidden**
A goal the system chose for itself carries the same “self” badge here as everywhere else — easy to audit which decisions were whose.

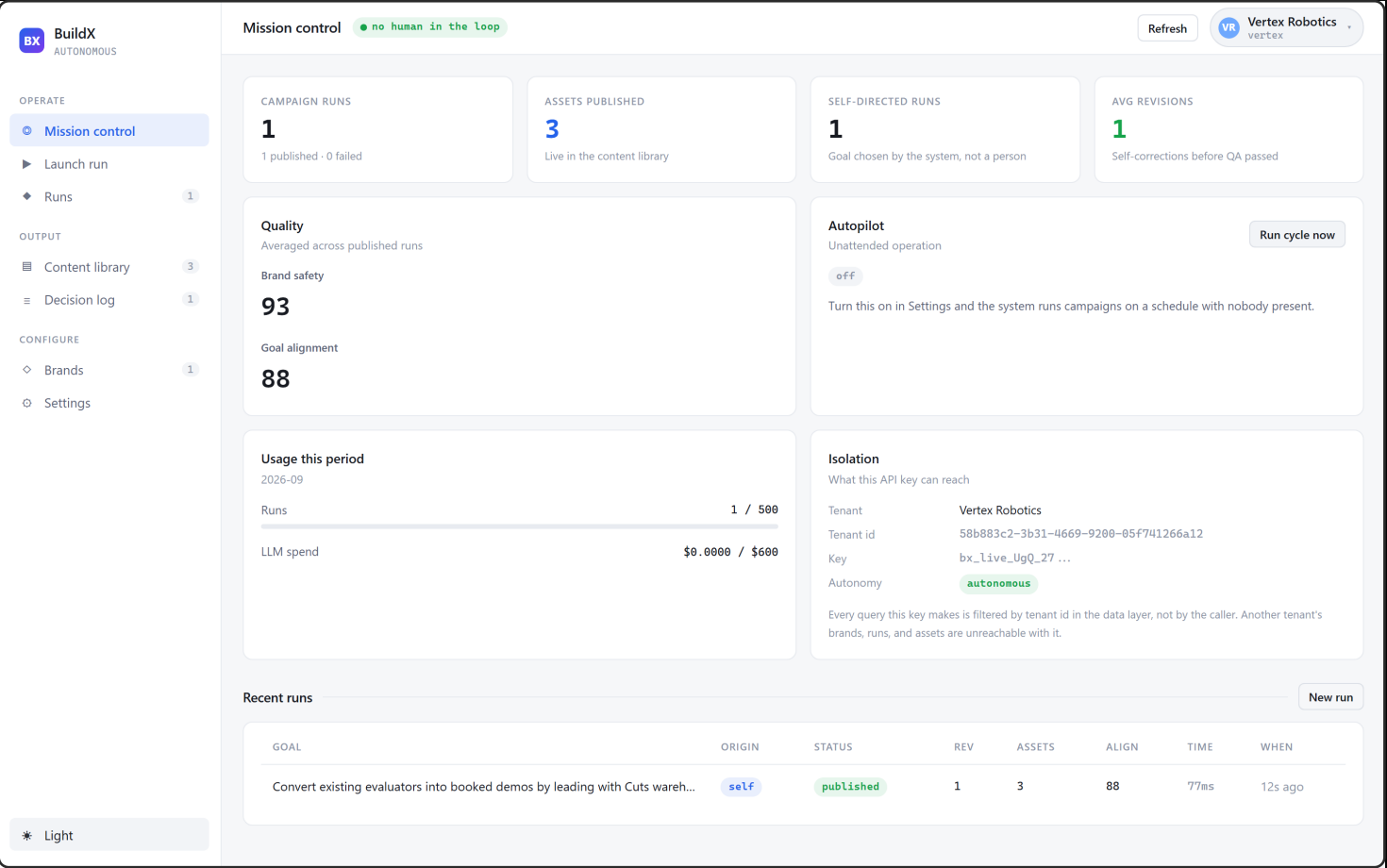
10 Settings — Autonomy Policy

CONFIG



Why it's built this way

- **Autonomy is a toggle, not a rebuild**
Flipping “Run fully autonomously” off doesn't change the agent pipeline at all — it just parks finished content as pending_review instead of publishing. One flag, same graph.
- **Guardrails are plain text, not a rules DSL**
Forbidden claims and banned phrases are one-per-line free text — QA is an LLM, so it can enforce a prose rule without a new schema every time policy changes.
- **The revision budget's consequence is spelled out inline**
The hint text under it explains arbitration in one sentence, right where you're setting the number — so changing it is an informed decision, not a guess.



Why it's built this way

- A full token swap, not an inversion filter**
 Every color on this screen is a deliberately chosen light-theme value — not a CSS filter over the dark palette — so contrast and hierarchy hold up exactly as intended.
- Status colors are re-tuned, not reused verbatim**
 Green/amber/red are darkened slightly for light backgrounds so they still meet contrast against white, rather than looking washed out.
- Same data, same layout, zero relearning**
 Nothing moves between themes — the only thing that changes is the palette, so switching mid-session costs the operator nothing.

What the design is actually arguing

Every screen in this deck exists to make one thing legible: a system is deciding things without you, and you can always see exactly what it decided and why. That is the whole design brief.

- **The revision banner**

is the single most important element in the product — it's the moment a human would normally intervene, made visible instead of invisible.

- **The isolation card**

on Mission Control exists so “multi-tenant” isn't a claim in a README — it's a fact the operator can see about their own key, every time they load the page.

- **The tenant switcher**

makes cross-tenant separation demonstrable in seconds: two keys, two completely disjoint datasets, one UI.

- **The decision log**

stands in for the approval trail a human reviewer would normally leave — nothing ships without a reason recorded.